

Double-click (or enter) to edit

```
pip install pandas numpy scikit-learn joblib
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.3)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from pyt
```

Upload the dataset or indicate the location to the file in your code

Phase 1: Explore the Dataset

First, let's see what we're working with.

Patient Admission Data ↓ Data Cleaning ↓ Feature Engineering ↓ Train/Test Split ↓ Logistic Regression ↓
Model Evaluation ↓ Feature Importance ↓ Saved Model (.pkl) ↓ Mobile Deployment (Future)

ebola-outcome-prediction/ | |— data/ | |— ESYADD_SCI_KT_ebola_master_database.csv | |—
notebooks/ | |— exploration.ipynb | |— models/ | |— src/ | |— train.py | |— evaluate.py |
|— predict.py | |— requirements.txt | |— README.md

```
import pandas as pd

df = pd.read_csv(
    r"ESYADD_SCI_KT_ebola_master_database.csv"
)

print("Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())

print("\nFirst rows:")
print(df.head())

print("\nMissing values:")
print(df.isnull().sum().sort_values(ascending=False).head(30))
```

```

4   triage                died negative                1   ...   NaN

      corr_calcium_mod3 chloride_mod3 magnesium_mod3 phosphate_mod3 \
0          NaN          NaN          NaN          NaN          NaN
1          NaN          NaN          NaN          NaN          NaN
2          NaN          NaN          NaN          NaN          NaN
3          NaN          NaN          NaN          NaN          NaN
4          NaN          NaN          NaN          NaN          NaN

      lactate_mod3 tco2_mod3 pt_mod3 aptt_mod3 inr_mod3
0          NaN      NaN      NaN      NaN      NaN
1          NaN      NaN      NaN      NaN      NaN
2          NaN      NaN      NaN      NaN      NaN
3          NaN      NaN      NaN      NaN      NaN
4          NaN      NaN      NaN      NaN      NaN

```

[5 rows x 262 columns]

Missing values:

```

inr_mod3                456
aptt_mod3               456
pt_mod3                 456
quininesulphate_med     456
promethazine_med        456
magnesium_mod3          456
phosphate_mod3          456
lactate_mod3            456
cotrimoxazole_med       456
chloride_mod3           456
tco2_mod3               456
quninedihydrochloride_med 456
aptt_mod2               456
pt_mod2                 456
neuro_hc                456
coma_hc                 456
inr_mod2                456
artesanate_amiodaquine_med 456
gentamicin_med          455
doxycycline_med         455
wbc_mod3                455
mcv_mod3                455
rbc_mod3                455
jaundice_ci             455
hct_mod3                455
platelets_mod3          455
mchc_mod3               455
mch_mod3                455
lympho_mod3             455
granulo_mod3            455
dtype: int64

```

Phase 2: Identify the Prediction Target

We need to find a column such as:

outcome final_outcome status died discharged survived

Once we know the actual column names, we can choose the target variable correctly.

```

death_labels = [
    "died positive",

```

```

        "died negative",
        "dead on arrival",
        "ebola+ survivor died before discharge",
        "died before test done"
    ]

    survive_labels = [
        "recovered and discharged"
    ]

    df = df[df["outcome"].isin(death_labels + survive_labels)]

    df["target"] = (
        df["outcome"]
        .isin(survive_labels)
        .astype(int)
    )

    print(df["target"].value_counts())

```

```

target
1    151
0    130
Name: count, dtype: int64

```

Step 3: Select Features

Use only admission variables.

```

features = [
    "age",
    "sex",
    "fever_ci",
    "vomit_nausea_ci",
    "diarrhoea_ci",
    "fatigue_ci",
    "abdominal_ci",
    "muscle_ci",
    "joint_ci",
    "headache_ci",
    "breathing_ci",
    "rash_ci",
    "bleeding_ci",
    "contact_ci",
    "funeral_ci",
    "travel_ci"
]

```

Phase 3: Build a Lightweight Model

For the showcase, I recommend:

Create training data:

```
X = df[features]
y = df["target"]
```

Step 4: Handle Missing Values

Simple baseline:

```
X = X.fillna(0)
```

Later we can improve.

Step 5: Train Logistic Regression

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

```
▼ LogisticRegression ⓘ ?
LogisticRegression(max_iter=1000)
```

Step 6: Evaluate

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, pred))
print("Precision:", precision_score(y_test, pred))
print("Recall:", recall_score(y_test, pred))
print("F1:", f1_score(y_test, pred))
```

```
Accuracy: 0.5087719298245614
Precision: 0.5555555555555556
Recall: 0.4838709677419355
F1: 0.5172413793103449
```

Step 7: Feature Importance

This is important for a healthcare project.

```
importance = pd.DataFrame({
    "feature": features,
    "coefficient": model.coef_[0]
})

importance = importance.sort_values(
    by="coefficient",
    ascending=False
)

print(importance)
```

	feature	coefficient
1	sex	0.678819
11	rash_ci	0.302351
14	funeral_ci	0.268188
4	diarrhoea_ci	0.244257
2	fever_ci	0.241229
9	headache_ci	0.215283
13	contact_ci	0.073095
7	muscle_ci	0.034909
8	joint_ci	-0.013630
0	age	-0.020103
10	breathing_ci	-0.076359
5	fatigue_ci	-0.085593
6	abdominal_ci	-0.162947
3	vomit_nausea_ci	-0.203194
15	travel_ci	-0.329793
12	bleeding_ci	-0.558283

Step 8: Save Model

```
import joblib

joblib.dump(model, "ebola_outcome_model.pkl")

['ebola_outcome_model.pkl']
```

because: tiny model size, fast, explainable, mobile-friendly

```
print(df["outcome"].value_counts())
```

outcome	
ebola negative	172
recovered and discharged	151
died positive	111
died negative	12
dead on arrival	5
non-recovered and referred	3
ebola+ survivor died before discharge	1

died before test done
Name: count, dtype: int64

1

Predict Patient Outcome

Predict Patient Outcome

Convert the target into:

Survived (1) recovered and discharged Died (0) died positive died negative dead on arrival ebola+ survivor died before discharge died before test done

Ignore:

non-recovered and referred

because the final outcome is unknown.